

PostgreSQL Data Recovery

Using WAL Archives and pg_wal Logs

When No Traditional Backup Images Are Available
Disaster Recovery Procedures for Production Environments

Target Audience	PostgreSQL DBAs, Infrastructure Engineers
PostgreSQL Versions	13, 14 (Log-Shipping) 16, 17 (Streaming + Slots)
Data Directory	/apps/pgsql_data/<version>
Log Directory	/apps/logs
Operating System	AlmaLinux 8 / 9
Classification	Disaster Recovery — Emergency Procedures

Table of Contents

1. Understanding WAL-Based Recovery — The Big Picture
2. Assess What You Have — Inventory and Triage
3. Scenario A — Damaged PGDATA + pg_wal Intact (Crash Recovery)
4. Scenario B — Old Base Backup + WAL Archive Chain (PITR)
5. Scenario C — Standby Server as Recovery Source
6. Scenario D — PGDATA Lost, WAL Archive Only (Partial Recovery)
7. Scenario E — pg_resetwal — Absolute Last Resort
8. WAL File Anatomy and Manual Inspection
9. Extracting Data from a Recovered or Partial Database
10. Log-Shipping vs. Streaming — Recovery Differences
11. Post-Recovery Validation Procedures
12. Prevention — Never Be Here Again

What You Have	Scenario	Recovery Possible?	Data Loss?
PGDATA intact but crashed + pg_wal intact	A — Crash Recovery	Yes — full	None (if WAL complete)
Old base backup + unbroken WAL archive chain	B — PITR	Yes — to any point in WAL chain	None to target point
Standby server running or recently stopped	C — Standby as Source	Yes — full or near-full	Minimal (replication lag)
PGDATA destroyed + WAL archive only (no base backup)	D — Partial Recovery	Partial — DDL and some data via pg_waldump	Significant
PGDATA corrupted, pg_wal missing/corrupt, no archive	E — pg_resetwal	Minimal — database starts but data may be inconsistent	Potentially high

2. Assess What You Have — Inventory and Triage

Before attempting any recovery, take a complete inventory of what assets survive. This determines which recovery scenario applies to your situation.

Step 1: Check Primary Data Directory

Inventory — Primary PGDATA:

```
# Check if PGDATA exists and is accessible
ls -la /apps/pgsql_data/<version>/

# Check key subdirectories
ls -la /apps/pgsql_data/<version>/base/ # Actual database files
ls -la /apps/pgsql_data/<version>/global/ # Shared catalog files
ls -la /apps/pgsql_data/<version>/pg_wal/ # Active WAL files
ls -la /apps/pgsql_data/<version>/pg_xact/ # Transaction status (was pg_clog)
ls -la /apps/pgsql_data/<version>/pg_multixact/ # Multi-transaction status

# Check control file (THE most important file in the cluster)
pg_controldata /apps/pgsql_data/<version>/

# Key fields to note from pg_controldata:
# Database cluster state: (shut down | in production | in archive recovery ...)
# Latest checkpoint location: 0/XXXXXXXX
# Latest checkpoint's TimeLineID: N
# Latest checkpoint's REDO location: 0/XXXXXXXX
# pg_control last modified: <timestamp>
```

Step 2: Check WAL Files

Inventory — WAL Files:

```
# Count and list active WAL files in pg_wal
ls -ltr /apps/pgsql_data/<version>/pg_wal/ | head -20
ls /apps/pgsql_data/<version>/pg_wal/ | wc -l

# Check WAL archive
ls -ltr /apps/pgsql_archives/ | head -20
ls /apps/pgsql_archives/ | wc -l
du -sh /apps/pgsql_archives/

# Find the OLDEST and NEWEST WAL files (determines recovery window)
ls /apps/pgsql_archives/ | sort | head -1 # Oldest
ls /apps/pgsql_archives/ | sort | tail -1 # Newest

# Check for timeline history files (indicate past promotions)
ls /apps/pgsql_archives/*.history 2>/dev/null
```

Step 3: Check for Any Base Backup (Even Old Ones)

Inventory — Base Backups:

```
# Search for any pg_basebackup outputs, pgBackRest repos, pg_dump outputs
find /apps/ -name 'backup_label' -type f 2>/dev/null
find /apps/ -name 'base.tar*' -type f 2>/dev/null
find /apps/ -name 'backup_manifest' -type f 2>/dev/null

# Check pgBackRest if installed
pgbackrest info 2>/dev/null

# Check pgBackRest stanza info
pgbackrest info 2>/dev/null

# Check for filesystem snapshots (LVM, ZFS)
lvs 2>/dev/null | grep snap
zfs list -t snapshot 2>/dev/null
```

Step 4: Check Standby / Replica Servers

Inventory — Standby Servers:

```
# Check if any standby servers exist and are accessible
ssh postgres@<standby_ip> "pg_controldata /apps/pgsql_data/<version>/"
ssh postgres@<standby_ip> "pg_ctl status -D /apps/pgsql_data/<version>/"

# If standby is running, check its status
ssh postgres@<standby_ip> \
"psql -U postgres -c 'SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn(),
now() - pg_last_xact_replay_timestamp() AS delay;'"
```

TIP: A running or recently-stopped standby is often the **best and fastest** recovery source. If one exists, jump to Scenario C immediately.

3. Scenario A — Damaged PGDATA + pg_wal Intact (Crash Recovery)

This is the most common and most recoverable scenario. The data directory exists but PostgreSQL crashed or was killed ungracefully. The pg_wal directory is intact with all required WAL files.

When This Applies

- Server crash (OOM kill, power failure, kernel panic)
- PostgreSQL process killed with SIGKILL or -m immediate shutdown
- Disk I/O errors during normal operation that did not destroy the data directory
- Filesystem went read-only due to disk errors, but data is still readable

Step A1: Assess the Damage

```
# Check control file state
pg_controldata /apps/pgsql_data/<version>/ | grep -E 'state|REDO|Timeline|checkpoint'

# Expected for a crash: 'in production' (was running when crash occurred)
# If 'shut down' = clean shutdown, just start normally
# If 'in archive recovery' = was a standby or mid-recovery when crashed
```

Step A2: Attempt Normal Startup (PostgreSQL's Built-in Crash Recovery)

PostgreSQL has automatic crash recovery. When it detects an unclean shutdown, it replays WAL from the last checkpoint's REDO location forward to reach a consistent state.

Crash Recovery Attempt:

```
# First: take a cold copy of PGDATA as safety net
rsync -a /apps/pgsql_data/<version>/ /apps/backups/pre_recovery_$(date +%Y%m%d)/

# Attempt startup – PostgreSQL will detect unclean shutdown and replay WAL
pg_ctl start -D /apps/pgsql_data/<version>/ -l /apps/logs/recovery_attempt.log

# Monitor recovery progress in real-time
tail -f /apps/logs/recovery_attempt.log
```

Expected Log Messages During Crash Recovery:

```
LOG: database system was interrupted; last known up at 2025-11-18 10:30:15 UTC
LOG: database system was not properly shut down; automatic recovery in progress
LOG: redo starts at 0/3000028
LOG: invalid record length at 0/3500A70: wanted 24, got 0
LOG: redo done at 0/3500A30 system usage: ...
LOG: checkpoint starting: end-of-recovery ...
LOG: checkpoint complete: ...
LOG: database system is ready to accept connections
```

NOTE: The line 'invalid record length ... wanted 24, got 0' is **normal** during crash recovery. It means PostgreSQL reached the end of valid WAL records. Recovery is complete at the prior record.

Step A3: If Normal Startup Fails — Missing WAL Files

If `pg_wal` is missing some files needed for crash recovery (e.g., they were recycled or the `pg_wal` directory was partially damaged), but they exist in the archive:

```
# Copy missing WAL files from archive to pg_wal
# Check which WAL file is needed from the error log:
# FATAL: could not open file "pg_wal/000000010000000000000003": No such file

cp /apps/pgsql_archives/000000010000000000000003 \
/apps/pgsql_data/<version>/pg_wal/
chown postgres:postgres /apps/pgsql_data/<version>/pg_wal/000000010000000000000003

# Retry startup
pg_ctl start -D /apps/pgsql_data/<version>/ -l /apps/logs/recovery_attempt.log
```

Alternatively, configure a `restore_command` so PostgreSQL can pull WAL from the archive automatically during recovery:

Using `restore_command` for Crash Recovery:

```
# Edit postgresql.conf to add restore_command before starting
echo "restore_command = 'cp /apps/pgsql_archives/%f %p'" \
>> /apps/pgsql_data/<version>/postgresql.conf

# Create recovery.signal (PG 12+) to enter non-standby recovery mode
touch /apps/pgsql_data/<version>/recovery.signal

# Start PostgreSQL – it will use restore_command to fetch missing WAL
pg_ctl start -D /apps/pgsql_data/<version>/ -l /apps/logs/recovery_attempt.log

# IMPORTANT: After recovery completes, remove restore_command from
# postgresql.conf if you don't want it permanently.
```

WARNING: Do NOT create `standby.signal` here — that would put the server in standby mode. Use `recovery.signal` (PG 12+) for one-time recovery that ends with the server becoming a normal primary after replay completes.

4. Scenario B — Old Base Backup + WAL Archive Chain (PITR)

You have found a base backup — even a very old one — and you have an unbroken WAL archive chain from that backup's start point to your desired recovery target. This is standard Point-in-Time Recovery (PITR).

CRITICAL: The WAL chain must be **unbroken** — every single WAL segment from the backup's start WAL to the target must exist in the archive. Even a single missing segment stops recovery at that point. Use the verification procedure below to check for gaps.

Step B1: Identify the Base Backup's Starting WAL

Identify Starting WAL:

```
# If you have a backup_label file from pg_basebackup:
cat /path/to/backup/backup_label

# Key fields:
# START WAL LOCATION: 0/2000028 (file 000000010000000000000002)
# CHECKPOINT LOCATION: 0/2000060
# BACKUP METHOD: streamed
# START TIME: 2025-10-01 02:00:00 UTC

# The 'file' value tells you the FIRST WAL file needed for recovery
# In this example: 000000010000000000000002
```

Step B2: Verify WAL Chain Continuity

WAL Chain Verification Script:

```
#!/bin/bash
# Script: verify_wal_chain.sh
# Checks for gaps in the WAL archive sequence

ARCHIVE_DIR="/apps/pgsql_archives"
START_WAL="000000010000000000000002" # From backup_label

echo "Checking WAL chain from $START_WAL ..."

# Extract timeline, high, low from WAL filename
TIMELINE=${START_WAL:0:8}
PREV_HIGH=""
PREV_LOW=""
GAPS=0

for wal_file in $(ls $ARCHIVE_DIR/${TIMELINE}* 2>/dev/null | sort); do
  fname=$(basename $wal_file)
  # Skip history files
  [[ "$fname" == *.history ]] && continue
  [[ "$fname" == *.backup ]] && continue

  HIGH=${fname:8:8}
  LOW=${fname:16:8}

  if [ -n "$PREV_HIGH" ]; then
    EXPECTED_LOW=$(printf '%08X' $((16#$PREV_LOW + 1)))
    if [ "$HIGH" = "$PREV_HIGH" ] && [ "$LOW" != "$EXPECTED_LOW" ]; then
      echo "GAP DETECTED after ${TIMELINE}${PREV_HIGH}${PREV_LOW}"
      echo " Expected: ${TIMELINE}${HIGH}${EXPECTED_LOW}"
      echo " Found: $fname"
      GAPS=$((GAPS + 1))
    fi
  fi
  PREV_HIGH=$HIGH
  PREV_LOW=$LOW
done

if [ $GAPS -eq 0 ]; then
  echo "WAL chain is CONTINUOUS — no gaps found"
  echo "Last WAL: ${TIMELINE}${PREV_HIGH}${PREV_LOW}"
else
  echo "WARNING: $GAPS gap(s) detected — recovery may be incomplete"
fi
```

Step B3: Restore the Base Backup

Restore Base Backup:

```
# IMPORTANT: Work in a SEPARATE directory – do NOT overwrite the current PGDATA
# in case it has recoverable data

# Create recovery target directory
mkdir -p /apps/pgsql_data/<version>_recovery

# Restore from pg_basebackup tar format:
tar xf /path/to/base.tar.gz -C /apps/pgsql_data/<version>_recovery/

# OR restore from pg_basebackup plain format (directory copy):
rsync -a /path/to/backup/ /apps/pgsql_data/<version>_recovery/

# Set ownership and permissions
chown -R postgres:postgres /apps/pgsql_data/<version>_recovery/
chmod 700 /apps/pgsql_data/<version>_recovery/
```

Step B4: Configure Recovery

For PostgreSQL 12+ (PG 13, 14, 16, 17):

PITR Configuration (PG 12+):

```
# Create recovery.signal (NOT standby.signal – we want a one-time recovery)
touch /apps/pgsql_data/<version>_recovery/recovery.signal

# Configure recovery in postgresql.conf or postgresql.auto.conf
cat >> /apps/pgsql_data/<version>_recovery/postgresql.auto.conf <<'EOF'

# Point to WAL archive
restore_command = 'cp /apps/pgsql_archives/%f %p'

# --- RECOVERY TARGET OPTIONS (choose ONE) ---

# Option 1: Recover to a specific point in time
# recovery_target_time = '2025-11-18 10:30:00 UTC'

# Option 2: Recover to the END of all available WAL (maximum data recovery)
# (Leave recovery_target unset and recovery.signal present)

# Option 3: Recover to a specific transaction ID
# recovery_target_xid = '12345678'

# Option 4: Recover to a named restore point
# recovery_target_name = 'before_migration'

# Option 5: Recover to the end of a specific WAL file
# recovery_target_lsn = '0/3500000'

# IMPORTANT: Include or exclude the target transaction
# recovery_target_inclusive = true # (default) include the target

# Follow latest timeline (important after promotions)
recovery_target_timeline = 'latest'

# Action after recovery: 'promote' makes it a writable primary (default in PG14+)
recovery_target_action = 'promote'
EOF
```

TIP: For **maximum data recovery** when you have no specific target time, leave `recovery_target` parameters unset. PostgreSQL will replay ALL available WAL from the archive and promote to a primary. This recovers the most data possible.

For PostgreSQL 11 and earlier — use `recovery.conf`:

```
cat > /apps/pgsql_data/<version>_recovery/recovery.conf <<'EOF'
restore_command = 'cp /apps/pgsql_archives/%f %p'
recovery_target_timeline = 'latest'
# recovery_target_time = '2025-11-18 10:30:00 UTC'
EOF
```

Step B5: Start Recovery

```
# Start PostgreSQL pointing to the recovery directory
pg_ctl start -D /apps/pgsql_data/<version>_recovery/ -l /apps/logs/pitr_recovery.log

# Monitor recovery progress
tail -f /apps/logs/pitr_recovery.log

# You will see messages like:
# LOG: starting archive recovery
# LOG: restored log file "000000010000000000000002" from archive
# LOG: restored log file "000000010000000000000003" from archive
# ... (one per WAL file – can be thousands)
# LOG: redo done at 0/5000A30
# LOG: last completed transaction was at log time 2025-11-18 10:29:55 UTC
# LOG: selected new timeline ID: 2
# LOG: archive recovery complete
# LOG: database system is ready to accept connections
```

NOTE: The log line 'last completed transaction was at log time ...' tells you exactly how far forward the recovery reached. This is the point to which your data has been recovered.

Step B6: Also Replay `pg_wal` from the Old Data Directory

If the old data directory's `pg_wal` has WAL files newer than what exists in the archive (the primary crashed before archiving its latest WAL), copy those files to the archive before starting recovery:

Recover Un-Archived WAL:

```
# Copy any WAL files from old PGDATA's pg_wal that are NOT in the archive
for f in /apps/pgsql_data/<version>/pg_wal/0000*; do
  fname=$(basename $f)
  if [ ! -f /apps/pgsql_archives/$fname ]; then
    echo "Copying un-archived WAL: $fname"
    cp $f /apps/pgsql_archives/
    chown postgres:postgres /apps/pgsql_archives/$fname
  fi
done

# These un-archived WAL files may contain the LAST few transactions
# that were committed but not yet archived – critical for maximum recovery!
```

5. Scenario C — Standby Server as Recovery Source

If a standby/replica server exists — running or recently stopped — it is almost always the fastest and most complete recovery source. The standby's data directory IS a continuous base backup that has been kept up-to-date through WAL replay.

Option C1: Promote the Standby to Primary

If the standby is healthy and you need the service back immediately:

```
# On Standby Server

# Check how far behind the standby is
psql -U postgres -c "SELECT now() - pg_last_xact_replay_timestamp() AS delay;"

# If acceptable, promote it
pg_ctl promote -D /apps/pgsql_data/<version>/

# Verify promotion
psql -U postgres -c "SELECT pg_is_in_recovery();" -- Must return 'f'
```

Option C2: Use Standby as a Base Backup Source for PITR

If you need to recover to a specific point in time, use the standby's data directory as a base backup combined with the WAL archive:

Standby as PITR Source:

```
# Stop the standby cleanly
pg_ctl stop -D /apps/pgsql_data/<version>/ -m fast

# Copy its data directory to a recovery location
rsync -a /apps/pgsql_data/<version>/ /apps/pgsql_data/<version>_recovery/

# Configure PITR recovery (same as Scenario B, Step B4)
touch /apps/pgsql_data/<version>_recovery/recovery.signal
# Remove standby.signal if present
rm -f /apps/pgsql_data/<version>_recovery/standby.signal

cat >> /apps/pgsql_data/<version>_recovery/postgresql.auto.conf <<'EOF'
restore_command = 'cp /apps/pgsql_archives/%f %p'
recovery_target_time = '2025-11-18 10:30:00 UTC'
recovery_target_timeline = 'latest'
recovery_target_action = 'promote'
EOF

# Start PITR recovery
pg_ctl start -D /apps/pgsql_data/<version>_recovery/ -l /apps/logs/pitr_recovery.log
```

LOG-SHIPPING (PG 13/14): PG 13/14 log-shipping standby: The standby may be behind by up to `archive_timeout` seconds. Before promoting, copy any remaining WAL files from the old primary's `pg_wal` to the archive, then let the standby replay them.

STREAMING (PG 16/17): PG 16/17 streaming standby: The standby is typically within seconds of the primary. Check `pg_stat_wal_receiver` for the last received LSN.

6. Scenario D — PGDATA Lost, WAL Archive Only (Partial Recovery)

CRITICAL: This is the **worst-case scenario**. You have WAL archive files but **no base backup and no data directory**. Full database recovery is **impossible** because WAL records describe changes to existing data pages — without the pages themselves, the changes cannot be applied. However, partial data extraction IS possible.

What You CAN Recover from WAL Files Alone

Recoverable	Method	Completeness
DDL statements (CREATE TABLE, ALTER, etc.)	pg_waldump — look for XLOG records with DDL markers	Good — can reconstruct schema
INSERT data (heap records)	pg_waldump — decode Heap/INSERT records	Partial — raw tuples, no column names without schema
Transaction commit timestamps	pg_waldump — XACT/COMMIT records	Complete
Sequence values	pg_waldump — Sequence records	Good
Full-page images (after checkpoint)	pg_waldump with FPW data	Partial — complete page snapshots exist but need interpretation

Step D1: Use pg_waldump to Extract Information

pg_waldump is the primary tool for reading WAL files. It decodes WAL records into human-readable format. It is included with every PostgreSQL installation.

pg_waldump Basics:

```
# Basic dump of a single WAL file
pg_waldump /apps/pgsql_archives/00000001000000000000000003

# Dump with full-page images (FPW) — shows complete page snapshots
pg_waldump -b /apps/pgsql_archives/00000001000000000000000003

# Dump a range of WAL files
pg_waldump -p /apps/pgsql_archives/ \
-s 0/3000000 -e 0/5000000

# Filter by resource manager (rmgr) — show only specific record types
pg_waldump -r Heap \
/apps/pgsql_archives/00000001000000000000000003

# Show only transaction commits and aborts
pg_waldump -r Transaction \
/apps/pgsql_archives/00000001000000000000000003
```

Step D2: Extract DDL (Schema Recovery)

Schema Recovery from WAL:

```
# DDL operations generate XLOG records with specific resource managers.
# Look for Standby/RUNNING_XACTS, XACT/COMMIT records near DDL,
# and full-page images of system catalogs.

# Dump all WAL files and search for DDL-related patterns
for f in /apps/pgsql_archives/0000000100*; do
pg_waldump $f 2>/dev/null
done | grep -i 'desc:' | grep -iE 'INSERT|UPDATE|DELETE' \
| grep -E 'rel 1259/|rel 1247/|rel 2615/|rel 2604/' > /apps/logs/ddl_records.txt

# rel OIDs for system catalogs:
# 1259 = pg_class (tables, indexes, sequences)
# 1247 = pg_type
# 2615 = pg_namespace (schemas)
# 2604 = pg_attrdef (column defaults)
# 1249 = pg_attribute (columns)

# This gives you hints about what tables were created/modified
# but not the exact DDL text (PostgreSQL doesn't store DDL text in WAL)
```

NOTE: PostgreSQL WAL does not contain literal SQL statements. It contains physical-level changes to data pages. Reconstructing the exact DDL requires interpreting catalog page changes, which is extremely labor-intensive.

Step D3: Extract INSERT Data from Heap Records**Extract Heap INSERT Data:**

```
# Extract Heap INSERT records – these contain the actual tuple data
pg_waldump -r Heap \
/apps/pgsql_archives/00000001000000000000000003 \
2>/dev/null | grep 'INSERT' > /apps/logs/heap_inserts.txt

# Sample output looks like:
# rmgr: Heap len (rec/tot): 59/59, tx: 745, lsn: 0/03001C20,
# desc: INSERT off 2 flags 0x00,
# blkref #0: rel 1663/16384/16385 blk 0

# The 'rel' value is: tablespace/database_oid/table_oid
# Without the catalog, you need to map table OIDs manually

# To see the actual data bytes, use -b flag for full-page images
pg_waldump -b -r Heap \
/apps/pgsql_archives/00000001000000000000000003 \
2>/dev/null | grep 'INSERT' > /apps/logs/heap_inserts_with_data.txt
```

Step D4: Extract Full-Page Images (FPW)

After each checkpoint, the first modification to a data page causes PostgreSQL to write a complete copy of that page to WAL (a Full-Page Write / Full-Page Image). These are complete snapshots of individual 8 KB data pages that can be decoded.

Extract Full-Page Images:

```
# Extract full-page images (FPIs) from WAL
pg_waldump -b /apps/pgsql_archives/00000001000000000000000003 \
2>/dev/null | grep 'FPW' > /apps/logs/fpw_records.txt

# Full-page images contain complete page snapshots including:
# - Page header with LSN, checksum, flags
# - Line pointers (item IDs) pointing to tuple data
# - Actual tuple data (column values in binary format)

# Advanced: Use pageinspect extension to decode if you can create
# a temporary database with the same schema elsewhere
```

Step D5: Reconstruct Data Using a Temporary Cluster

If you can reconstruct the schema (from application code, ORM migrations, documentation, or another environment), you can create a new empty cluster and use WAL data to populate it:

Reconstruct with Temporary Cluster:

```
# Strategy: Create a new empty cluster with the same schema,
# then use pg_waldump output to reconstruct INSERT data.

# 1. Initialize a new cluster
initdb -D /apps/pgsql_data/<version>_temp/ --data-checksums

# 2. Start it and create the database with the same schema
pg_ctl start -D /apps/pgsql_data/<version>_temp/
psql -U postgres -c "CREATE DATABASE recovered;"
psql -U postgres -d recovered -f schema_from_application.sql

# 3. Parse pg_waldump output to extract tuple data
# and convert to INSERT statements
# This requires custom scripting to:
# - Map table OIDs to table names
# - Decode binary tuple data according to column types
# - Generate INSERT statements

# This is extremely labor-intensive but may be the only option
# for critical data that exists nowhere else.
```

7. Scenario E — pg_resetwal — Absolute Last Resort

LAST RESORT: `pg_resetwal` is a **destructive, last-resort tool**. It forcibly resets the WAL system, allowing a database with a corrupted or missing `pg_wal` and/or `pg_xact` directory to start. However, it **may cause data loss, silent data corruption, and referential integrity violations**. Use **ONLY** when all other recovery methods have failed.

When `pg_resetwal` Is Appropriate

- The data directory exists but PostgreSQL refuses to start due to missing/corrupt WAL
- The `pg_xact` (transaction commit status) directory is damaged
- The control file is corrupted and you can find no other way to start
- No base backup exists, no standby exists, and you need to extract whatever data you can
- You have already exhausted Scenarios A through D

Step E1: Make a Complete Copy FIRST

```
# ABSOLUTELY MANDATORY: Copy the entire data directory before touching anything
rsync -a /apps/pgsql_data/<version>/ /apps/backups/pre_resetwal_$(date +%Y%m%d)/

# This copy is your ONLY safety net. pg_resetwal modifies the data directory in place.
```

Step E2: Examine the Current State

```
# Try to read the control file
pg_controldata /apps/pgsql_data/<version>/ 2>&1

# If pg_controldata fails, the control file is corrupt.
# pg_resetwal will recreate it.

# Check what WAL files exist
ls -ltr /apps/pgsql_data/<version>/pg_wal/

# Check pg_xact (transaction status) directory
ls -la /apps/pgsql_data/<version>/pg_xact/
```

Step E3: Run `pg_resetwal`

`pg_resetwal` Dry Run:

```
# DRY RUN FIRST – shows what it would do without modifying anything
pg_resetwal -n /apps/pgsql_data/<version>/

# Review the output carefully:
# Current pg_control values:
# ...current values...
# Values to be changed:
# First log segment after reset: 0000000100000000000000006
# NextXID: 0:750
# ...etc...
```

Execute pg_resetwal:

```
# If the dry run looks reasonable, execute for real:
pg_resetwal /apps/pgsql_data/<version>/

# If pg_xact directory is missing or corrupt:
# pg_resetwal will warn that transaction status data is missing.
# Transactions that were in-progress at crash time will be marked as committed
# by default – this is the source of potential data corruption.

# If it complains about missing pg_xact files, manually create empty ones:
mkdir -p /apps/pgsql_data/<version>/pg_xact/
# Then retry pg_resetwal
```

Step E4: Start the Database and Extract Data Immediately

Start and Extract:

```
# Start PostgreSQL
pg_ctl start -D /apps/pgsql_data/<version>/ -l /apps/logs/resetwal_start.log

# IMMEDIATELY dump all data you can
pg_dumpall -U postgres -f /apps/backups/emergency_dumpall_$(date +%Y%m%d).sql

# Or dump specific critical databases
pg_dump -U postgres -Fc -f /apps/backups/critical_db.dump critical_db

# Check for obvious corruption
psql -U postgres -c "SELECT datname FROM pg_database;"
```

CRITICAL: After pg_resetwal, **do NOT trust this database for production**. Some transactions that were in-progress may appear committed. Some committed transactions may have partial data. Indexes may be inconsistent with heap data. Extract all data via pg_dump, then restore into a fresh cluster and validate thoroughly.

Step E5: Validate Data After pg_resetwal

Data Validation:

```
# Run REINDEX on all databases to fix potentially corrupt indexes
psql -U postgres -c "REINDEX SYSTEM postgres;"
psql -U postgres -d <your_db> -c "REINDEX DATABASE <your_db>;"

# Check for heap corruption using amcheck extension
psql -U postgres -d <your_db> -c "CREATE EXTENSION IF NOT EXISTS amcheck;"
psql -U postgres -d <your_db> -c "
SELECT bt_index_check(c.oid)
FROM pg_index i
JOIN pg_class c ON c.oid = i.indexrelid
WHERE c.relam = (SELECT oid FROM pg_am WHERE amname = 'btree');"

# Check for orphaned rows or missing foreign key references
# (Application-specific queries needed here)
```

8. WAL File Anatomy and Manual Inspection

WAL File Naming Convention

```
WAL filename: 000000010000000000000003
|-----||-----||-----|
Timeline High Low
(8 hex) (8 hex) (8 hex)

Timeline 1, Segment 3 = 000000010000000000000003
Timeline 2, Segment 3 = 000000020000000000000003 (after promotion)

Timeline history files: 00000002.history
Contains: parentTLI switchLSN reason
Example: 1 0/3000060 no recovery target specified
```

pg_waldump Resource Managers (rmgr) Reference

rmgr	What It Records	Recovery Value
Heap	INSERT, UPDATE, DELETE, HOT_UPDATE on table rows	HIGH — contains actual data
Heap2	CLEAN, FREEZE, VISIBLE operations	Low — maintenance operations
Btree	B-tree index insertions, splits, deletions	Medium — index changes
Transaction	COMMIT, ABORT, PREPARE for 2PC	HIGH — transaction boundaries
XLOG	Checkpoint, end-of-recovery, parameter change, FPI	HIGH — recovery control
Standby	Running transactions, invalidation messages	Medium — standby support
Sequence	Sequence value changes (nextval calls)	HIGH — sequence state
Storage	CREATE, TRUNCATE operations on storage	HIGH — schema changes
RelMap	Relation mapping file changes	Medium — catalog mapping
Database	CREATE DATABASE, DROP DATABASE	HIGH — database-level DDL
Tablespace	CREATE/DROP TABLESPACE	HIGH — tablespace DDL
MultiXact	Multi-transaction ID allocations	Low — internal bookkeeping

Practical pg_waldump Filtering Examples

Useful pg_waldump Filters:

```
# Show only data-modifying operations (INSERT/UPDATE/DELETE)
pg_waldump -r Heap /apps/pgsql_archives/000000010000000000000003 \
| grep -E 'INSERT|UPDATE|DELETE|HOT_UPDATE'

# Show commits with timestamps (identify when transactions happened)
pg_waldump -r Transaction /apps/pgsql_archives/000000010000000000000003 \
| grep 'COMMIT'

# Show operations on a specific relation (table) by OID
# First, find your table's OID if you know the database OID
pg_waldump /apps/pgsql_archives/000000010000000000000003 \
| grep 'rel 1663/16384/16385' # tablespace/db_oid/rel_oid

# Show full-page writes (complete page snapshots)
pg_waldump -b /apps/pgsql_archives/000000010000000000000003 \
| grep 'FPW'

# Filter by transaction ID (find all changes in a specific transaction)
pg_waldump /apps/pgsql_archives/000000010000000000000003 \
| grep 'tx: 745,'

# Count records by type (understand what the WAL contains)
pg_waldump /apps/pgsql_archives/000000010000000000000003 \
2>/dev/null | awk '{print $2}' | sort | uniq -c | sort -rn
```

9. Extracting Data from a Recovered or Partial Database

Once you have a database running (from any scenario), immediately extract all data. A recovered database should be treated as fragile — get the data out to a safe format.

Priority 1: Full Dump

```
# Dump ALL databases (roles, tablespaces, and data)
pg_dumpall -U postgres -f /apps/backups/recovered_dumpall.sql

# OR dump specific databases in custom format (faster, compressed)
pg_dump -U postgres -Fc -Z5 -f /apps/backups/critical_db.dump critical_db

# For very large databases, use directory format for parallel dump
pg_dump -U postgres -Fd -j4 -f /apps/backups/critical_db_dir/ critical_db
```

Priority 2: Critical Table Extraction

```
# If full dump is too slow or partially failing, extract critical tables first
psql -U postgres -d <your_db> -c "\COPY important_table TO '/apps/backups/important_table.csv' CSV
HEADER"

# Or use pg_dump for specific tables
pg_dump -U postgres -t important_table -Fc -f /apps/backups/important_table.sql <your_db>

# List all tables to prioritize extraction
psql -U postgres -d <your_db> -c "
SELECT schemaname, tablename,
pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS size
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;"
```

Priority 3: Handle Corrupt Tables

Handling Corrupt Tables:

```
# If pg_dump fails on specific tables due to corruption:

# Step 1: Set zero_damaged_pages to skip corrupt pages
psql -U postgres -d <your_db> -c "SET zero_damaged_pages = on;"

# Step 2: Try to read the table
psql -U postgres -d <your_db> -c "
SET zero_damaged_pages = on;
\COPY important_table TO '/apps/backups/important_table.csv' CSV HEADER;"

# Step 3: For tables with corrupt indexes, try sequential scan only
psql -U postgres -d <your_db> -c "
SET enable_indexscan = off;
SET enable_bitmapscan = off;
SET zero_damaged_pages = on;
SELECT * FROM important_table;"

# Step 4: If specific blocks are corrupt, skip them by reading ranges
psql -U postgres -d <your_db> -c "
SET zero_damaged_pages = on;
SELECT * FROM important_table WHERE ctid >= '(0,1)' AND ctid < '(1000,0)';"
```

WARNING: `zero_damaged_pages = on` causes PostgreSQL to return zero-filled data for corrupt pages instead of throwing an error. This means some rows will silently contain wrong or missing data. Use this only for emergency extraction — always validate the extracted data.

10. Log-Shipping vs. Streaming — Recovery Differences

Recovery Aspect	Log-Shipping (PG 13/14)	Streaming + Slots (PG 16/17)
WAL availability	Archive directory is the SOLE WAL source. If archive is lost, recovery window is limited to pg_wal on surviving servers.	WAL is in archive + retained by replication slots on primary. Slots prevent WAL recycling, so a connected standby often has all needed WAL.
Maximum data loss	Up to archive_timeout seconds of un-archived WAL. The last partial WAL segment (not yet switched) is NOT archived.	Sub-second — streaming is continuous. Un-archived WAL exists in pg_wal on the primary and on the standby.
Standby as recovery source	Standby has all WAL that was successfully archived and restored. May be behind by minutes.	Standby has all WAL up to last streaming receive point. Typically seconds behind.
pg_waldump access	Archive directory has complete WAL history. May span months or years if not cleaned.	Archive has history + slot retains recent WAL. pg_wal on primary has un-archived segments.
Un-archived WAL recovery	CRITICAL: The last WAL segment (between last archive and crash) exists ONLY in pg_wal on the primary. Copy it to archive before starting PITR.	Same applies, but the standby may also have this WAL segment via streaming.
Monitoring for gaps	Check pg_stat_archiver for failed_count. Compare last_archived_wal with actual files in archive directory.	Check pg_stat_replication for lag. Replication slot ensures no WAL recycling while standby is connected.

Critical: Un-Archived WAL — The Last-Mile Problem

In **both** log-shipping and streaming setups, PostgreSQL only archives a WAL segment when it is complete (16 MB). The current, partially-filled WAL segment is **never archived** (unless archive_timeout forces a switch). This means:

- If the primary crashes, the last partial WAL segment exists ONLY in the primary's pg_wal directory
- This segment may contain the last few minutes to hours of transactions, depending on write volume
- For log-shipping standbys (PG 13/14): this segment is NOT on the standby at all
- For streaming standbys (PG 16/17): this segment WAS streamed in real-time, so the standby has it
- During any recovery, always copy the primary's pg_wal files to the archive before starting PITR

Recover Un-Archived WAL (Run Before Any PITR):

```
# ALWAYS run this before starting PITR recovery – recovers the 'last mile'
for f in /apps/pgsql_data/<version>/pg_wal/0000*; do
fname=$(basename $f)
[[ "$fname" == *.partial ]] && continue # Skip partial files for now
if [ ! -f /apps/pgsql_archives/$fname ]; then
echo "Recovering un-archived WAL: $fname"
cp $f /apps/pgsql_archives/
chown postgres:postgres /apps/pgsql_archives/$fname
fi
done

# Also check for .partial WAL files – these are incomplete segments
# They contain valid WAL records up to the crash point
ls /apps/pgsql_data/<version>/pg_wal/*.partial 2>/dev/null
# If found, rename by removing .partial suffix and copy to archive:
for f in /apps/pgsql_data/<version>/pg_wal/*.partial; do
fname=$(basename $f .partial)
cp $f /apps/pgsql_archives/$fname
chown postgres:postgres /apps/pgsql_archives/$fname
done
```

11. Post-Recovery Validation Procedures

Step 1: Basic Health Check

```
# Verify the database is accepting connections
psql -U postgres -c "SELECT version();"
psql -U postgres -c "SELECT datname, pg_size_pretty(pg_database_size(datname)) FROM pg_database;"

# Check for errors in the log
grep -iE 'error|fatal|panic|corrupt' /apps/logs/postgresql-*.log | tail -20
```

Step 2: Data Integrity Check

```
# Check all tables are accessible
psql -U postgres -d <your_db> -c "
SELECT schemaname, tablename,
(SELECT count(*) FROM pg_catalog.pg_class c2
WHERE c2.oid = (schemaname||'.'||tablename)::regclass) AS exists
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema');"

# Row counts for critical tables (compare with known values if available)
psql -U postgres -d <your_db> -c "
SELECT schemaname || '.' || relname AS table,
n_live_tup AS estimated_rows
FROM pg_stat_user_tables
ORDER BY n_live_tup DESC;"

# Run ANALYZE to update statistics
psql -U postgres -d <your_db> -c "ANALYZE VERBOSE;"
```

Step 3: Index Integrity Check

```
# Install amcheck extension for B-tree index verification
psql -U postgres -d <your_db> -c "CREATE EXTENSION IF NOT EXISTS amcheck;"

# Check all B-tree indexes (non-destructive verification)
psql -U postgres -d <your_db> -c "
SELECT bt_index_check(c.oid, heapallindexed => true)
FROM pg_index i
JOIN pg_class c ON c.oid = i.indexrelid
WHERE c.relam = (SELECT oid FROM pg_am WHERE amname = 'btree');"

# If any indexes fail, rebuild them
psql -U postgres -d <your_db> -c "REINDEX DATABASE <your_db>;"
```

Step 4: Determine Exact Recovery Point

```
# Check the last transaction timestamp
psql -U postgres -d <your_db> -c "
SELECT max(xmin::text::bigint) AS last_xid FROM pg_class;"

# Check the recovery log for the exact recovery endpoint
grep -i 'last completed transaction' /apps/logs/pitr_recovery.log
grep -i 'redo done at' /apps/logs/pitr_recovery.log

# Compare with application-level timestamps in your data
psql -U postgres -d <your_db> -c "
SELECT max(updated_at) AS last_known_timestamp FROM important_table;"
```

12. Prevention — Never Be Here Again

The fact that you needed this guide means your backup and DR strategy had gaps. Here is how to ensure you never face this situation again.

The Three Pillars of PostgreSQL Disaster Recovery

Pillar	What	How	Verification
1. Regular Base Backups	Periodic snapshots of the entire cluster	pg_basebackup, pg_dump, pgBackRest, or LVM/ZFS snapshots on a schedule	Test restore monthly. Verify backup_label + WAL chain.
2. Continuous WAL Archiving	Every WAL segment archived to durable storage	archive_command with error checking. NFS, S3, or dedicated backup server.	Monitor pg_stat_archiver for failed_count > 0. Alert immediately.
3. Standby Replication	Live copy of the database on a separate server	Log-shipping (PG 13/14) or streaming+slots (PG 16/17).	Monitor replication lag. Test promotion procedure quarterly.

Minimum Viable Backup Configuration

Minimum postgresql.conf for DR:

```
# postgresql.conf – MINIMUM settings for disaster recovery

wal_log_hints = on # Required for pg_rewind
wal_level = replica # Required for replication and PITR
archive_mode = on # Enable WAL archiving

# archive_command that FAILS LOUDLY (returns non-zero) if copy fails
archive_command = 'test ! -f /apps/pgsql_archives/%f && cp %p /apps/pgsql_archives/%f'

# For PG 13/14 log-shipping: force WAL switch periodically
archive_timeout = 300 # 5 minutes max data loss window

# For PG 16/17 streaming: cap slot WAL retention
max_slot_wal_keep_size = 50GB # Prevent disk fill from stale slots
```

Automated Backup Verification Script

Daily Backup Health Check:

```
#!/bin/bash
# backup_health_check.sh – Run daily via cron

ARCHIVE_DIR="/apps/pgsql_archives"
ALERT_EMAIL="dba@company.com"

# 1. Check archiver is working
FAILED=$(psql -U postgres -t -c \
"SELECT failed_count FROM pg_stat_archiver;" | xargs)
if [ "$FAILED" -gt 0 ]; then
echo "ALERT: $FAILED WAL archive failures" | \
mail -s "PG Archive Alert" $ALERT_EMAIL
fi

# 2. Check archive is recent (within last 30 min)
LAST_ARCHIVE=$(psql -U postgres -t -c \
"SELECT extract(epoch FROM now() - last_archived_time) \
FROM pg_stat_archiver;" | xargs)
if [ $(echo "$LAST_ARCHIVE > 1800" | bc) -eq 1 ]; then
echo "ALERT: No WAL archived in 30+ minutes"
fi

# 3. Check base backup is recent (within last 7 days)
LATEST_BACKUP=$(find /apps/backups/ -name 'backup_label' \
-mtime -7 2>/dev/null | wc -l)
if [ "$LATEST_BACKUP" -eq 0 ]; then
echo "ALERT: No base backup in last 7 days"
fi
```

CRITICAL: The single most important rule: **Test your restores regularly.** A backup that has never been tested is not a backup — it is a hope. Schedule monthly restore tests on a separate server and verify data integrity.

End of Guide